

MARKET MICROSTRUCTURE SIMULATOR

*A Limit-Order-Book Market Built From Scratch, With Agent-Based Order Flow,
Empirical Validation Against Microstructure Theory, and a
Reinforcement-Learning Market-Making Agent Benchmarked Against
Avellaneda-Stoikov*

TOBY EDWARDS

Portfolio Project, Technical Report, August 2026

github.com/tobiedwards01/marketmicrostructuresim

Results	25
Summary	26
8 Conclusions	27
What Drives Spreads, In This Simulator	27
Each Market Maker's Biggest Risk	27
What I Would Change About the Model	27
Closing	28
A Appendix	29
Technology Stack	29
Repository Structure	29
Reproducing These Results	29
Repository	30

1

Introduction & Motivation

This project builds a limit-order-book market from scratch, populates it with several types of trading agents, and studies the market dynamics that fall out of their interaction, things like bid-ask spreads, price impact, and adverse selection. It then replaces a naive market maker with an agent that learns its own quoting strategy through reinforcement learning, and checks what it learned against a closed-form analytical benchmark from the market-making literature.

The Pitch

Real exchanges match buy and sell orders under price-time priority. Real market makers quote two-sided prices and manage the risk of holding inventory while informed traders try to pick off stale quotes. This project is a miniature, from-scratch version of that world: a matching engine that enforces the same rules a real exchange enforces, a population of agents whose incentives create realistic order flow, and, at the centre of it, a market maker that has to learn to survive in that world the same way a real one would.

Why This Project

This project targets finance and quant roles, specifically trading, quantitative research, and market making. It is a compressed version of what market-making and execution research teams actually do day to day: build simulators, test strategies against realistic order flow, and reason carefully about adverse selection and inventory risk. The goal was not to describe these ideas but to implement them, run them, and see what actually happens, including the places where the textbook theory did not cleanly match what the simulator produced.

What This Project Demonstrates

- **Systems design.** An event-driven simulation with a matching engine whose correctness was treated as the highest-risk, most test-intensive part of the build.
- **Market microstructure theory, applied.** Not just described: adverse selection (Glosten-Milgrom), informed trading and price impact (Kyle), and optimal market making (Avellaneda-Stoikov) are all implemented, not just cited.
- **Reinforcement learning applied to a genuinely hard sequential decision problem.** Balancing spread capture, adverse selection, and inventory risk with no explicit model of any of the three, learned purely from a profit-and-loss reward signal.

- **Rigorous empirical validation.** Comparing the simulator's own emergent behaviour against known theoretical and empirical results, and reporting honestly where the match was only partial.
- **Technical communication.** This report, and the running design and decision log kept throughout the build (see Appendix A).

How the Project Was Structured

The build was split into eight phases, each with its own concrete deliverable, its own test coverage, and its own entry in a running decisions log recording what was chosen, why, and what alternative was considered. This report follows that same phase structure. Sections 2 through 7 correspond directly to Phases 0 and 1 through 6 of the build, and Section 8 draws conclusions across all of them.

Phase	Deliverable	Report Section
Phase 0: Research & Design	Core data model and event-loop design (DESIGN.md)	2
Phase 1: Core Matching Engine	Price-time-priority matching engine with a full test suite	2
Phase 2: Baseline Agent Population	Noise traders, an informed trader, a naive market maker	3
Phase 3: Simulation & Metrics	Event loop instrumentation and a metrics pipeline	4
Phase 4: Emergent Behaviour Analysis	Three stylized facts checked against theory	5
Phase 5: Learning Market-Making Agent	A trained RL agent vs. the Avellaneda-Stoikov benchmark	6
Phase 6: Stress Test	A flash-crash case study across three market-maker designs	7
Phase 7: Writeup & Portfolio Report	This document	n/a

SCOPE DISCIPLINE

Two things were deliberately cut, and both are flagged here rather than silently skipped. One is a formal hypothesis-based property test suite, since a randomized invariant test over 500 operations was judged to cover the same ground. The other is validation against real LOBSTER tick data, a separate and meaningfully sized piece of work in its own right, and it's noted as the strongest candidate for extension in Section 8.

2

Matching Engine Design

Phases 0 & 1, Research & Design, Core Matching Engine

Before writing any code, the core data model and event-loop design were sketched in DESIGN.md. That covered the matching engine's data structures, the algorithm for walking the book, and the open design questions that needed resolving before implementation could begin. This section summarizes what was built and, more importantly, the specific design decisions that turned out to matter.

Core Data Model

Two data types anchor the whole simulator. An **Order** carries an id, an owning agent id, a side (buy or sell), a type (limit or market), a price in integer ticks, an original quantity and a mutable remaining quantity, a submission timestamp, a monotonically increasing sequence number, and a status (new, partially filled, filled, or cancelled). A **Trade** is an immutable record of one fill: price, quantity, the maker and taker order and agent ids, and the aggressor side, which is the side of the order that crossed the spread. That last field turns out to be the single most useful one in the entire data model for downstream analysis.

WHY INTEGER TICKS, NOT FLOATS

Every price is stored as an integer number of ticks, where one tick equals one cent, rather than as a floating-point dollar amount. Comparing floats for price-level equality is a classic source of matching-engine bugs, since 0.1 plus 0.2 does not equal 0.3 in binary floating point. Representing price as an integer from the start removes that whole category of bug before it can occur, and conversion to a dollar figure happens only at the display boundary.

Order Book Structure

The book is two sorted maps, one per side, each mapping a price to a first-in-first-out queue of resting orders at that price. It's implemented with the sortedcontainers library's SortedDict, which gives $O(\log n)$ price-level insertion and $O(1)$ best-price lookup. A separate dictionary maps order id to order object for $O(1)$ cancellation lookups. Best bid, best ask, spread, and mid price are all read properties computed from the book's current state.

The Matching Algorithm

An incoming order sweeps the opposite side of the book while a price-eligibility predicate holds. That's always true for market orders, and for limit orders it holds while the best opposing price still crosses the incoming order's own limit price. At each price level it fills against the oldest eligible resting order first, which is price-time priority, decrementing both sides' remaining quantity and emitting a Trade. A resting order that reaches zero remaining quantity is marked filled and removed from its queue. An incoming order with quantity left over after the book is exhausted either rests, if it's a limit order, or has its remainder cancelled immediately, if it's a market order. That second behaviour is standard immediate-or-cancel semantics, since a resting market order isn't really a concept that exists on any real exchange.

Three Decisions Worth Highlighting

- **Cancellation uses lazy deletion.** A cancelled order is marked cancelled in place rather than removed from its queue immediately, and the matching loop skips past cancelled orders, purging them, when it next reaches them. This is simpler to implement correctly than an eagerly maintained doubly-linked list, at the cost of needing to explicitly account for cancelled-but-not-yet-purged orders anywhere book state is read. That cost surfaced as a real, if minor, correctness bug later, covered in the callout below.
- **Self-trade prevention skips rather than cancels.** When an incoming order would only be able to match a resting order from the same agent, the matching loop skips past that resting order, continuing on to the next order at that price level or the next price level, rather than cancelling either side. This is more forgiving than most real exchanges, which typically cancel one or both sides outright. It's a deliberate simplification, documented as such, and its consequences (a book that can briefly report a locked price from one agent's own crossed quotes) were later traced directly to a real, observable artifact in the simulator's output, described in Section 3.
- **Market orders never rest.** This is immediate-or-cancel semantics: fill whatever is available, then cancel the rest instantly. That also produces a useful signal on its own, since the fraction of a market order that goes unfilled during a liquidity shortage is exactly what the Phase 6 stress test wants to observe.

A REAL BUG THIS DESIGN SURFACED

Best-bid and best-ask were originally implemented as a direct read of the top of each price queue. Because cancellation is lazy, this could report a price with nothing tradeable actually resting there, since a cancelled order can still be sitting at the front of the queue. This wasn't a hypothetical concern. It was caught in Phase 2 when the naive market maker cancelled its own stale quote and immediately asked for the current mid price to decide where to requote, and got a stale answer back. The fix was to have best-bid and best-ask actively purge cancelled orders from the front of the queue before reporting a price, a small and well-contained fix once the lazy-deletion contract was made to apply consistently everywhere the book state is read, not only inside the matching loop itself.

The Event Loop

The simulator is a discrete-event simulation: a min-heap of timestamp, sequence number, and agent id tuples, popped in time order. The sequence number is a single shared counter used both to break timestamp ties in the event queue and as every order's own sequence field. That one counter serving two purposes is what makes "what happened at exactly this simulated instant, and in what order" always fully deterministic and reproducible, which matters a great deal once simulation runs are being compared to each other, as in Phase 4, or replayed for a stress-test scenario, as in Phase 6.

Testing Philosophy

The matching engine was treated, deliberately, as the single highest-risk component in the entire project, the piece most likely to hide a subtle bug that would quietly corrupt every downstream metric. It received the most concentrated testing effort: hand-constructed cases for every fill scenario (full fill, partial fill, multi-level sweep, price-time priority among same-price orders), explicit self-trade-prevention cases, and, as a final hardening pass, a 500-step randomized test that submits a mixed sequence of orders and cancellations across five agents and checks, after every single step, that no two different agents' resting orders could have crossed without matching. It also runs a global conservation check at the end, confirming that total filled quantity across all orders equals twice the total traded volume.

3

Agent Population & Architecture

Phase 2, Baseline Agent Population

Five distinct agent types were built over the course of this project, each with a clearly scoped role. This section describes the architecture that lets any of them trade in the same market, then walks through each agent in turn.

Shared Architecture

Every agent implements two methods. One is **next_wake_time**, which tells the event loop when it should next be woken. Agents typically use a Poisson arrival process, an exponentially distributed inter-arrival time, for realistic, bursty order flow. The other is **act**, which is called when the agent wakes and may submit or cancel orders through a narrow **MarketAccess** interface: submit a limit order, submit a market order, cancel an order, and read the current book state. Each agent owns its own seeded random-number generator, so an entire simulation run is exactly reproducible given a fixed set of per-agent seeds. Every result in this report was produced this way, and every number quoted below was reproduced from a fresh run immediately before this report was written.

The three market-making agents, the naive one, Avellaneda-Stoikov, and the learned agent, share a further common base, **QuotingAgent**. It was introduced midway through the project once it became clear the same machinery, cancelling the previous two-sided quote, reconciling fills and cash against the shared trade log, and respecting an inventory limit, was about to be duplicated a third time. Each concrete market maker now implements only the pricing decision itself, and the base class owns everything else, including tracking each agent's own running cash and inventory from the shared trade log. That matters because a resting quote can be filled by another agent at any later wake, and there's no way for the agent to know about it except by checking the trade log the next time it wakes, so relying only on the immediate return value of its own order submissions wouldn't be enough.

Noise Trader: Zero-Intelligence Liquidity

Modelled on the classic Gode & Sunder (1993) zero-intelligence trader. At each wake it submits one random limit order, with the side chosen uniformly, the price a random offset from the current live mid price (falling back to a fixed reference price only when the book is empty), and the quantity drawn from a configured range. It cancels its own previous order first, so it holds exactly one live order at a time, replaced every period, and has no view of value beyond wherever the market currently is. Five independent noise traders, each with a different random seed, form the baseline liquidity and order-flow generator for every simulation in this project.

WHY ONE LIVE ORDER, REPLACED EACH WAKE, MATTERS

An earlier version let noise-trader orders accumulate forever. Two visible problems followed once the metrics pipeline in Phase 3 could actually chart them. Order-book depth grew without bound over a session, and, because each trader was now quoting around a drifting mid price rather than a fixed one, an old, never-cancelled order could end up priced on the wrong side of a trader's own new order. That produced a genuinely negative bid-ask spread whenever self-trade prevention correctly refused to match the two, as described in the self-trade-prevention decision in Section 2. Matching the actual textbook zero-intelligence model, one live order per trader, fixed both problems at once.

Informed Trader: The Source of Adverse Selection

Has noisy private access to a random-walk true price that the rest of the market cannot observe directly. At each wake it steps its belief of the true price by a Gaussian increment, forms a noisy signal of it, and compares that signal to the book's current best bid and ask. If the signal sits far enough above the best ask, it buys aggressively with a market order, and if it sits far enough below the best bid, it sells. It trades using market orders specifically because an informed trader with a real, if temporary, information edge wants immediate execution before that edge decays, rather than posting passive liquidity and waiting. This is the single mechanism in the whole simulator responsible for adverse selection, since it systematically picks off quotes that have gone stale relative to where the market maker's beliefs should have moved.

Naive Market Maker: The Baseline Strategy

Quotes a fixed half-spread around the current mid price, or a reference price if the book is empty, re-quoting on a fixed interval, and simply stops quoting a side once its inventory limit on that side is reached. There's no skewing and no reaction to time or volatility. It's the simplest possible market-making strategy, and it's the baseline every later agent is measured against.

A LIMIT THAT IS NOT QUITE A LIMIT

Because the inventory check only gates whether to submit a new quote, not how large to size it, a single full-size fill landing exactly at the boundary can carry inventory a little past the configured cap. This was observed directly in a real run: with a configured 50-unit cap, one Phase 2 run finished with an inventory of 51, and the Phase 6 stress test finished with 57. It was left as-is deliberately, since it's a genuine, useful finding about naive strategies rather than a bug to silently patch. See Section 7 for more.

Avellaneda-Stoikov Market Maker: The Analytical Benchmark

Implements the closed-form optimal quoting strategy from Avellaneda & Stoikov (2008), "High-Frequency Trading in a Limit Order Book." Two equations define it completely:

```
reservation price = mid price - inventory × gamma × sigma-squared × (time remaining)
optimal spread = gamma × sigma-squared × (time remaining) + (2 / gamma) × ln(1 +
gamma / k)
```

Here gamma is the market maker's risk aversion, sigma is the mid price's volatility, time remaining is the distance to a fixed horizon, and k controls how quickly order-flow intensity decays with distance from the mid price. Two behaviours fall directly out of the formulas that the naive strategy simply cannot produce. First, the reservation price shifts away from the mid price in the direction that reduces inventory, so a long position pushes both quotes down and a short position pushes them up. Second, the spread widens with more time remaining to the horizon, since there's more time for inventory risk to hurt, and narrows as the horizon approaches. Sigma is estimated from realized data, a real simulated run's mid-price volatility, rather than guessed. Gamma and k are hand-tuned so the resulting spread lands in a realistic range for this market, since there's no equally direct way to estimate either one from data. That's a documented simplification, not an oversight.

Q-Learning Market Maker: The Learned Strategy

A tabular Q-learning agent, deliberately kept simple, following the project's own guidance to start with tabular methods and reach for deep reinforcement learning only if the state space genuinely needs it, which a fifteen-state, twenty-five-action problem does not. Full design detail is in Section 6. In brief, its state is a discretized pair of an inventory bucket and a time-remaining bucket, its action is a discretized pair of a spread choice and a skew choice deliberately mirroring Avellaneda-Stoikov's own two decision variables, and its reward at each decision is the change in its own mark-to-market profit and loss since the previous decision. That means inventory risk becomes a real, experienced cost rather than a hand-tuned penalty term bolted on separately.

Agent Summary

Agent	Role	Key Mechanism
NoiseTrader	Baseline liquidity / order flow	One live order, replaced each wake, no view on value
InformedTrader	Adverse selection source	Private noisy signal on a latent true price; trades via market orders
NaiveMarketMaker	Baseline market-making strategy	Fixed spread, hard inventory cap, no skewing
AvellanedaStoikovMarketMaker	Analytical benchmark	Closed-form reservation price + spread from inventory, volatility, horizon

Agent	Role	Key Mechanism
QLearningMarketMaker	Learned strategy	Tabular Q-learning; reward = change in mark-to-market P&L

4

Simulation & Metrics Framework

Phase 3, Simulation Framework & Metrics

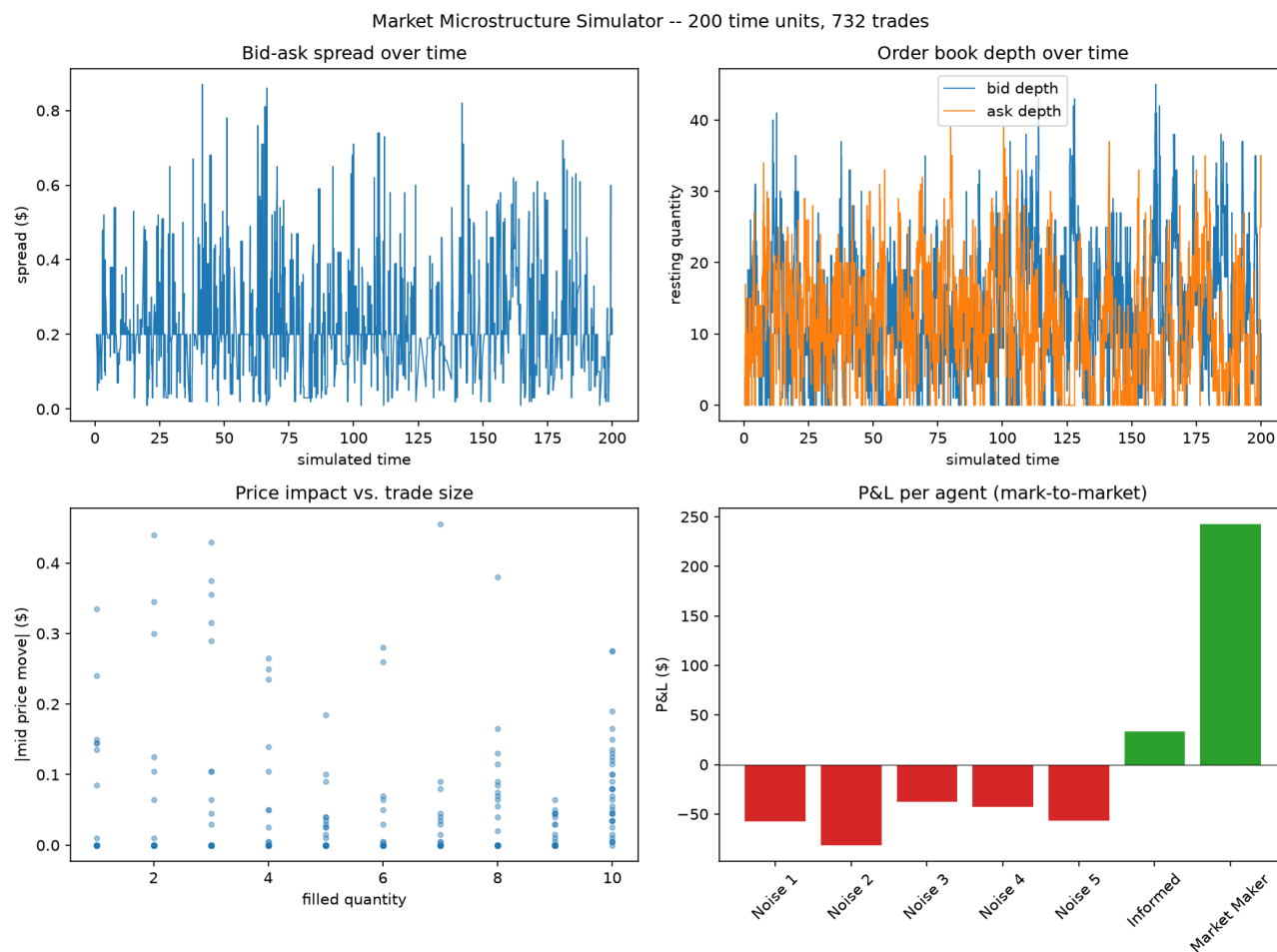
With agents generating realistic order flow, the event loop was instrumented to record everything needed for empirical analysis automatically, as a side effect of ordinary trading, with no separate logging pass or after-the-fact reconstruction required.

What Gets Recorded

- **Book snapshots.** Best bid, best ask, spread, mid price, and live depth on both sides, recorded automatically after every order submission or cancellation, giving spread and depth a complete time series rather than only being readable at whatever moment someone happens to ask.
- **Order impact.** The mid-price move caused by one order submission, covering both trade-driven impact, where a fill sweeps through resting liquidity, and quote-driven impact, where a new best price is posted without necessarily trading, since both genuinely move a real market.
- **Trade log.** Every individual fill, from which mark-to-market profit and loss per agent is derived: cash flow from fills, plus ending inventory valued at a mark price, combining realized and unrealized profit and loss into one number without needing per-lot cost-basis tracking.
- **Order book depth.** Total live resting quantity per side, explicitly excluding orders that are cancelled but not yet purged from their queue, the lazy-deletion gap flagged in Section 2, since a naive sum over the raw queues would silently overcount actual liquidity.

The Dashboard

A single simulated session, with five noise traders, one informed trader, and one naive market maker, run for two hundred simulated time units, produces the four-panel dashboard below: spread over time, order book depth over time, price impact against trade size, and mark-to-market profit and loss per agent.



A single simulated session's metrics dashboard.

THE clearest result in the whole project

Every one of the five noise traders loses money, while the informed trader and the market maker both profit. This is the textbook adverse-selection story appearing directly in the P&L panel with no further analysis needed. Informed order flow and disciplined market-making both extract value from uninformed liquidity, exactly as Glosten-Milgrom and Kyle both predict.

5

Empirical Validation: Stylized Facts

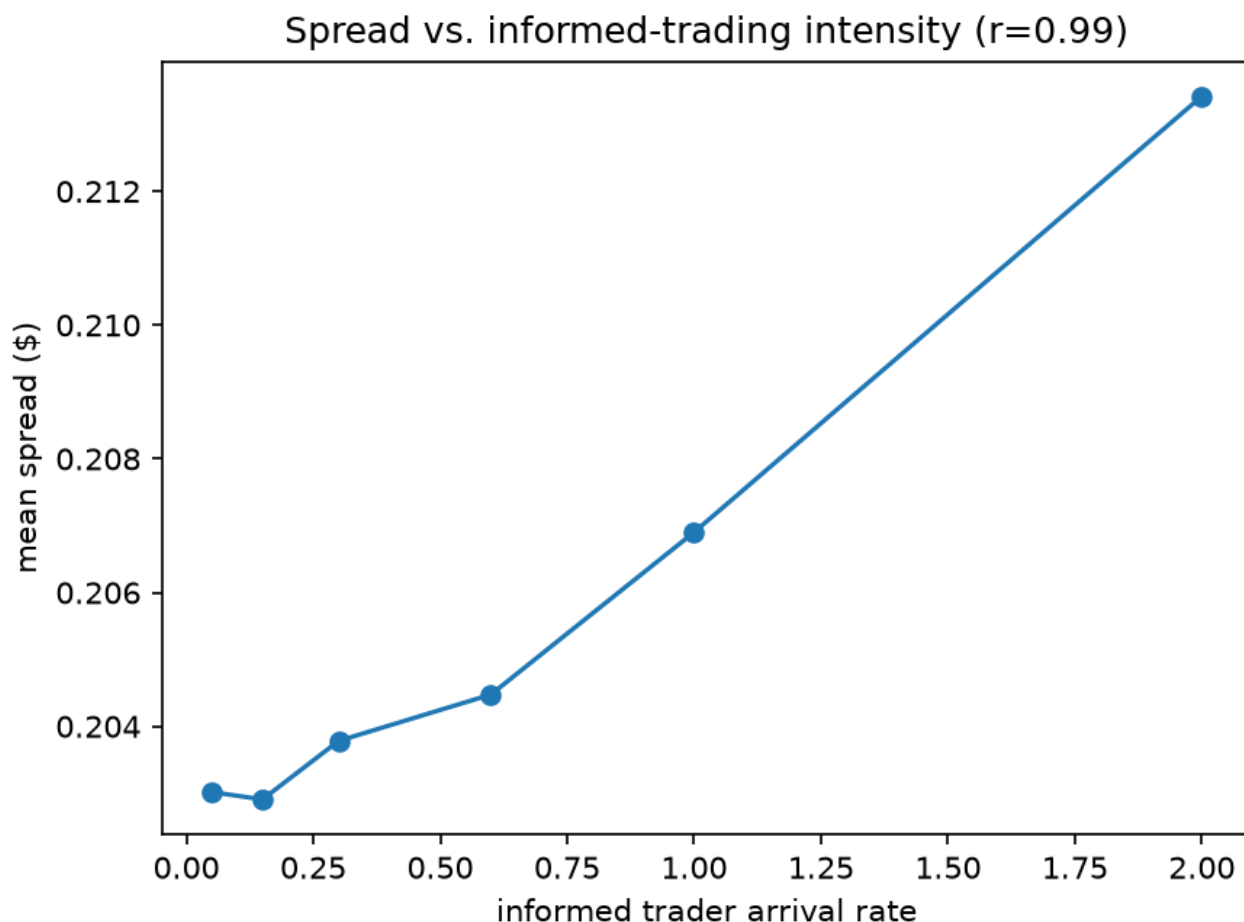
Phase 4, Emergent Behaviour Analysis

Three stylized facts from market microstructure theory were checked against the simulator's own output, each with its own controlled experiment, its own chart, and an explicit citation of the theoretical result being tested. All numbers below are from a real run, reproduced immediately before this report was finalized.

1. Spread Widens With Informed-Trading Intensity

Theory (Glosten & Milgrom, 1985). A market maker who cannot distinguish informed order flow from noise prices that risk into the spread. The more likely an incoming order is to come from someone with better information, the wider the market maker must quote to avoid being picked off on average.

Method. Every noise-trader and market-maker parameter was held fixed while the informed trader's arrival rate was swept across six levels, from 0.05 through 2.0, with three replicate two-hundred-time-unit runs at each level using the same three seed sets throughout, for a fair paired comparison. Mean spread was measured from the automatically recorded book snapshots.



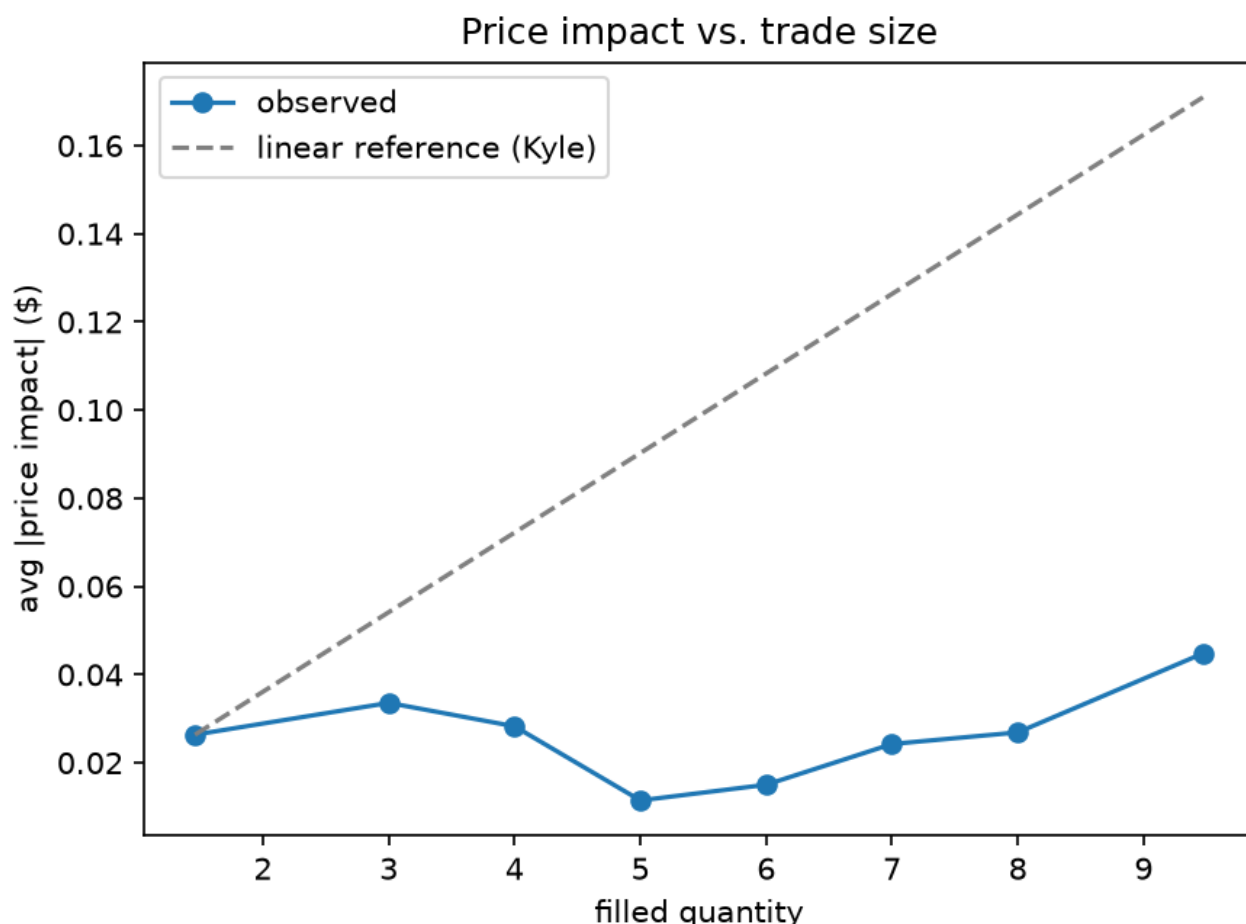
Mean spread vs. informed-trading intensity.

Result: matches theory. Pearson correlation between informed arrival rate and mean spread was 0.989. The effect is modest in absolute size, roughly a five percent wider spread across a forty-times range in informed intensity, but it's monotonic and clearly not noise. The modest size has a clear explanation: the naive market maker used in this experiment quotes a fixed spread and does not reason about adverse-selection risk at all, so the widening observed is a second-order effect of faster quote depletion, not a first-order pricing response. A market maker that explicitly prices adverse selection, Avellaneda-Stoikov in Section 6, would be expected to show a substantially stronger version of this effect.

2. Price Impact Is Concave in Trade Size

Theory (Kyle, 1985). Kyle's model derives linear price impact as the equilibrium outcome of an informed trader optimally hiding order flow in noise, and that's the textbook baseline. A large body of later empirical and theoretical work, including the market-impact "square-root law", instead finds impact that grows sub-linearly, or concave, in trade size in real markets.

Method. One simulation run's filled orders were bucketed by size into eight equal-width buckets, with mean absolute mid-price impact computed per bucket from the automatically recorded order-impact log.



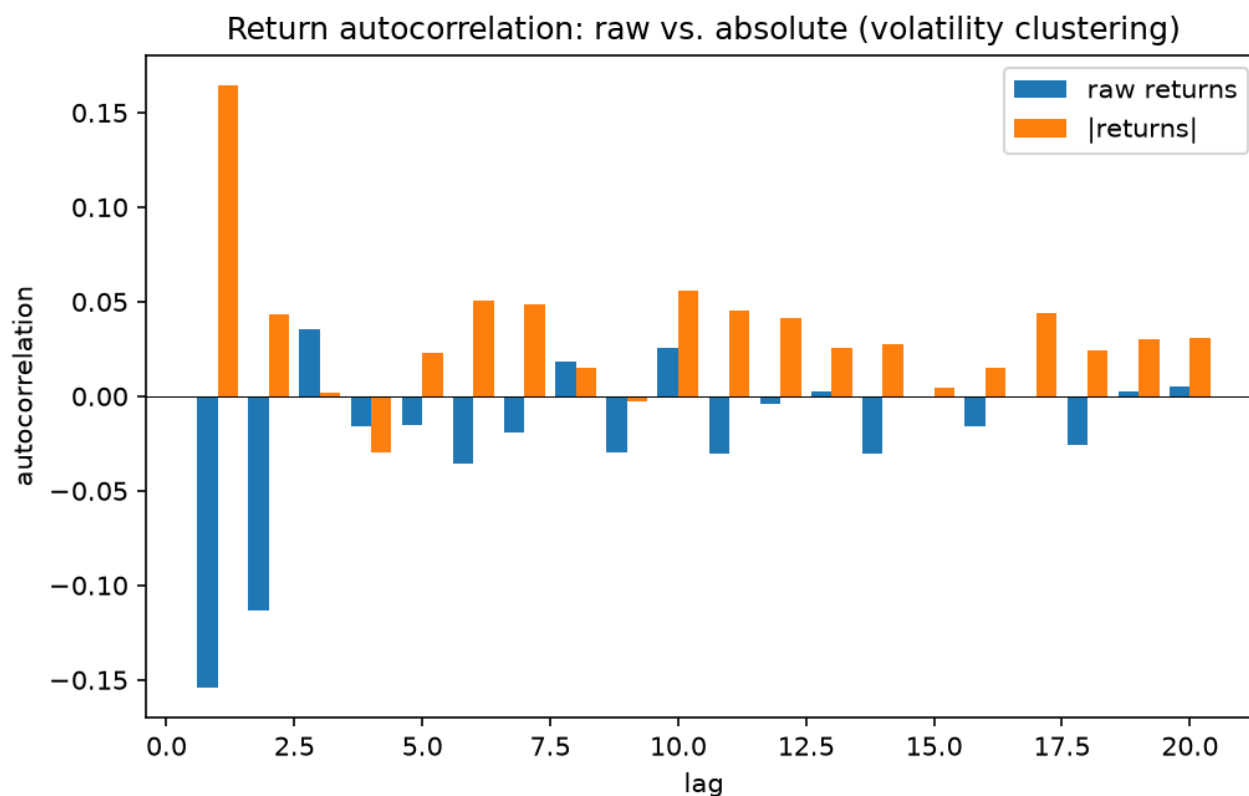
Price impact vs. trade size, against Kyle's linear reference.

Result: broadly concave, but noisy. Concavity was checked as whether impact per unit of size falls as size grows, since if so, marginal impact is shrinking, which is the definition of concavity. The correlation between size and impact-per-unit-size was -0.767 , consistent with concavity, and the chart shows observed impact sitting well below the linear Kyle reference across the whole size range, never catching up as size grows. One bucket, size five, is a visible non-monotonic dip, and per-bucket sample sizes range from thirty-three to a hundred, which is enough to support the qualitative claim of sub-linear, not linear, but not a precise functional form.

3. Volatility Clustering

Theory (Mandelbrot, 1963; Cont, 2001). One of the most robust stylized facts of real asset returns is that large price moves tend to be followed by more large moves, of either sign, and small moves by small ones. This shows up as clearly positive autocorrelation in absolute returns, persisting across many lags, in

sharp contrast to raw signed returns, whose autocorrelation stays close to zero.



Return autocorrelation: raw vs. absolute, lags 1 through 20.

Result: partial support, not a clean match. There's a real, positive lag-one signal in absolute returns, at 0.165, so some short-lived clustering is genuinely present. But it decays to essentially zero by lag two and stays there, while real markets typically show this clustering persisting across dozens or hundreds of lags. This simulator does not reproduce the strong, persistent form of the stylized fact, and the reason is mechanistic rather than a shortfall of data. Nothing in the simulator has time-varying volatility built into it, since the informed trader's true-price process uses a constant volatility parameter for the whole run, so there's no mechanism for quiet and turbulent periods to alternate. It's also worth noting that raw returns show a negative, not near-zero, autocorrelation at short lags, a mean-reversion signature most likely coming from the market maker actively pulling price back toward its own quotes on every requote, and from noise traders re-centring on the live mid price each time they wake.

Summary

Stylized fact	Theory	Result
Spread widens with informed intensity	Glosten-Milgrom (1985)	Matches ($r = 0.989$)
Price impact is concave in size	Kyle (1985) linear baseline	Broadly matches ($r = -0.767$)
Volatility clustering	Mandelbrot (1963) / Cont (2001)	Weak: lag 1 only, explained mechanistically

Two of three stylized facts hold up cleanly, and the third holds up partially, with a specific, mechanistic explanation rather than an unexplained gap. That's a more useful result for a technical interview than three clean checkmarks would be, since it demonstrates the simulator was actually interrogated rather than declared successful on the first pass.

WHAT WASN'T DONE

The brief's optional stretch goal, sanity-checking against real LOBSTER tick data, was not attempted. It requires acquiring and licensing-checking an external dataset and choosing a comparable instrument and time window, which is a meaningfully separate piece of work in its own right, not an extension of what's here. See Section 8 for why it's the strongest candidate for future work.

6

Reinforcement-Learning Market-Making Agent

Phase 5, Learning Market-Making Agent

The naive market maker, with its fixed spread and no inventory skewing, is replaced here with two things: a closed-form analytical benchmark, Avellaneda-Stoikov, already introduced in Section 3, and a tabular Q-learning agent trained purely on a profit-and-loss reward signal, with no knowledge of the Avellaneda-Stoikov formula at all. The central question this phase answers is whether an agent that only ever observes whether its own mark-to-market profit and loss went up or down can rediscover anything resembling the textbook-optimal quoting behaviour.

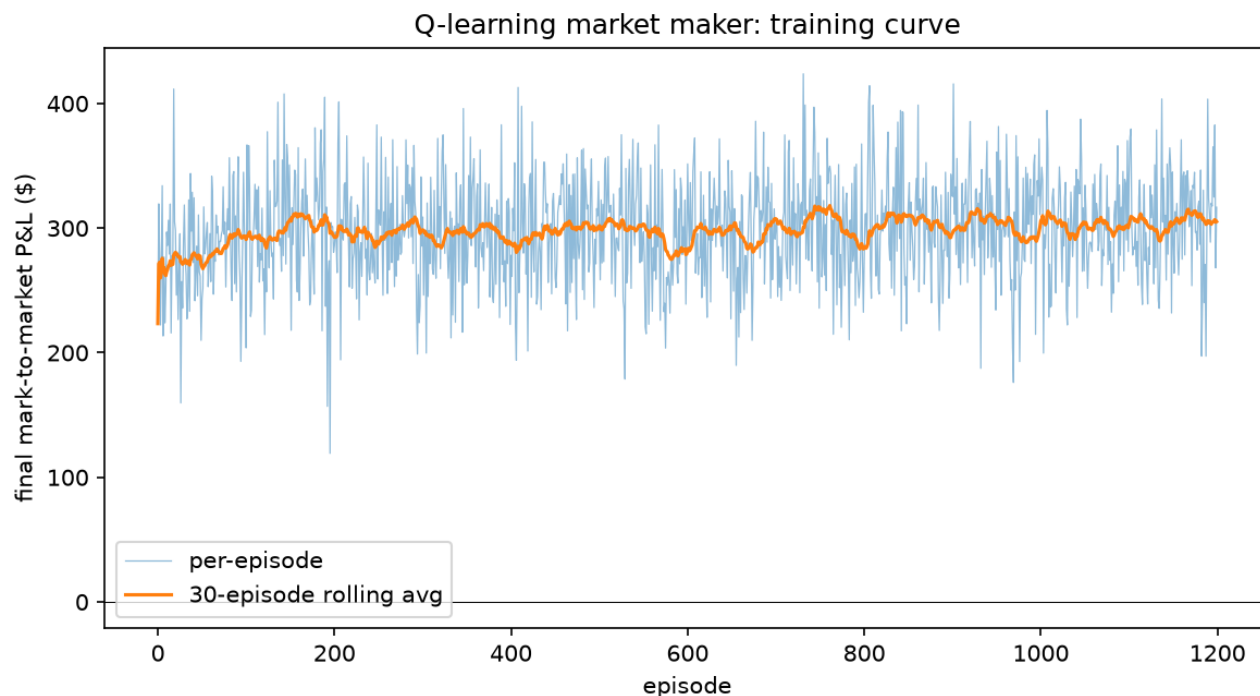
State, Action, and Reward

State is a discretized pair: an inventory bucket, with five buckets spanning very short to very long relative to the agent's inventory limit, and a time-remaining bucket, with three buckets for early, mid, and late in the episode. That's fifteen states in total. Action is a discretized pair too, a half-spread choice from five values and a skew choice from five values, giving twenty-five actions, deliberately built to mirror Avellaneda-Stoikov's own two decision variables, with spread driven by time-to-horizon and skew driven by inventory, so the eventual comparison between what the agent learns and the closed form is genuinely apples-to-apples rather than comparing unrelated representations. Reward at each decision is the change in the agent's own mark-to-market profit and loss since the previous decision, so inventory risk is a real, experienced cost the agent has to learn to manage, rather than a hand-tuned penalty term layered on separately.

Training used a standard off-policy Q-learning update, taking the maximum over the next state's action values rather than the actually chosen next action, with a constant learning rate, epsilon-greedy exploration decaying from 0.3 to a floor of 0.02 over training, and ties among equally good actions broken randomly rather than always toward the first index. Tabular Q-learning was chosen deliberately over a deep-learning approach, following the project's own guidance to start simple and reach for a neural network only if the state space genuinely needs it, and fifteen states and twenty-five actions do not.

Training

Trained for 1,200 episodes against the same order flow, five noise traders and one informed trader, that every other market-making agent in this project has faced. Performance converged to a stable average mark-to-market profit and loss of roughly \$290 to \$310 per two-hundred-time-unit episode within about fifty episodes, and held there for the remaining eleven hundred with no divergence or collapse.



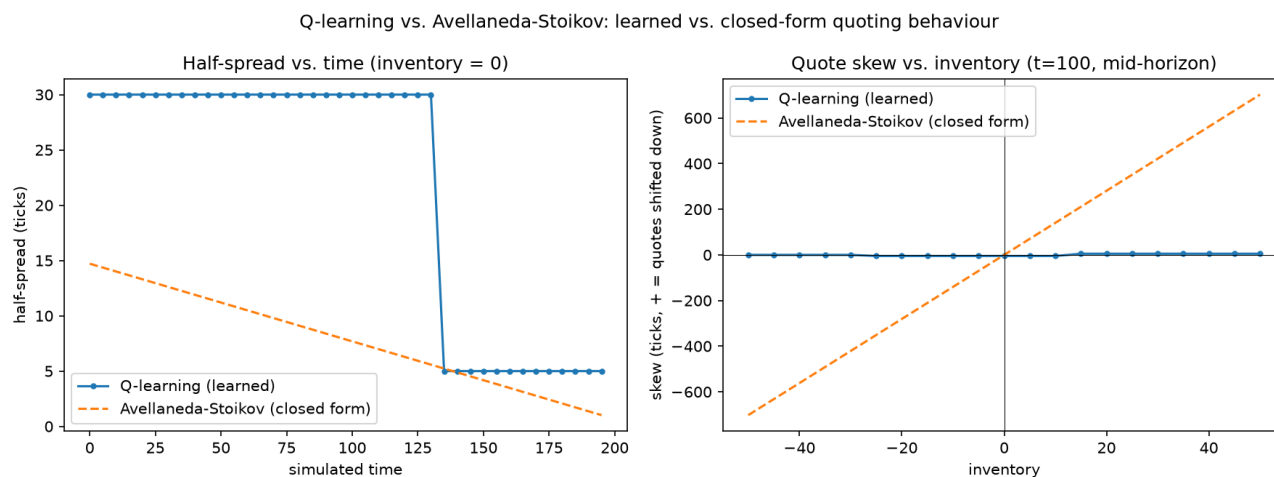
Training curve: final mark-to-market P&L per episode, 1,200 episodes.

A False Alarm Worth Describing

The first trained model, after six hundred episodes, produced a skew-versus-inventory relationship that looked backwards from what theory predicts, with quotes barely moving with inventory, and what little movement there was pointing the wrong way. Before writing that up as the agent having learned something genuinely different from theory, a per-state visit-count tracker was added to check whether it was simply an under-exploration artifact. Extreme-inventory states, reached less often under a policy that is actively trying to stay near flat, seemed like the obvious suspect. The counts refuted that theory directly: all fifteen states received between twenty-three thousand and forty-six thousand visits across six hundred episodes, roughly uniform, which is not the signature of rare-state neglect at all.

The model was retrained for 1,200 episodes without changing anything else, and the relationship corrected itself to the theoretically expected sign. The most likely explanation is that Q-learning here uses a constant learning rate rather than a decaying one, so the table never fully settles to a fixed point. Aggregate performance had already plateaued by episode one hundred, but the fine-grained, genuinely low-signal distinction of exactly which skew choice is marginally better than its neighbours kept needing more updates to resolve cleanly. This is reported here, rather than quietly presenting only the corrected second run, because explaining that a first hypothesis turned out to be wrong, and what the cause actually was, is a more honest and more useful account of the process than a single clean result would be.

Learned vs. Closed-Form Quoting Behaviour



Q-learning's learned quotes vs. Avellaneda-Stoikov's closed form, across the same state grid.

Simulated time	Q-learning half-spread	AS half-spread
0	30.0	14.7
40	30.0	11.9
80	30.0	9.1
120	30.0	6.3
160	5.0	3.5

Both narrow as the horizon approaches, the learned agent in one discrete step because of its coarse three-bucket time discretization, and Avellaneda-Stoikov smoothly and continuously. This is a genuine result. The learned agent was never told that spread should shrink near the end of an episode, and only ever observed a profit-and-loss number, yet it rediscovered the direction of the closed-form prediction from reward alone, landing in the same rough order of magnitude on both sides.

Inventory	Q-learning skew	AS skew
-50	0.0	-702.2
-10	-5.0	-140.4
0	-5.0	0.0
10	-5.0	140.4
50	5.0	702.2

Direction matches on skew too, since both push quotes down when long and up when short, but magnitude diverges sharply. Avellaneda-Stoikov's skew at maximum inventory is roughly a hundred and forty times larger than the learned agent's. This isn't really a case of the learned agent failing to discover what theory

says is correct. Avellaneda-Stoikov's skew term is linear and mathematically unbounded in inventory, so at fifty units of inventory in this simulated market it prescribes moving quotes by roughly seven dollars on a hundred-dollar instrument, which isn't a quote anyone would actually post, since at that point it is not making a market so much as refusing to trade. This is a well-documented practical critique of the raw Avellaneda-Stoikov formula, and real trading desks typically cap or dampen the inventory term. The learned agent, constrained to a small, human-designed action grid, never explores anything close to that scale, which arguably makes it a more realistic constraint than the closed form imposes on itself.

SUMMARY

The learned agent independently rediscovered that spread should narrow near a horizon and that quotes should skew away from inventory in the correct direction, purely from a profit-and-loss reward signal, with no access to the Avellaneda-Stoikov formula. It did not rediscover the closed form's specific magnitudes for inventory skew, and the closed form's own unbounded linear term is arguably the less realistic of the two once real quote sizes are considered.

7

Stress Test: Flash-Crash Case Study

Phase 6, Stress Test

A mini flash-crash case study. A sudden, large drop in the informed trader's belief of the true price partway through an otherwise ordinary session, simulating unexpected bad news, gives a way to look at how each of the three market makers built across this project holds up against it.

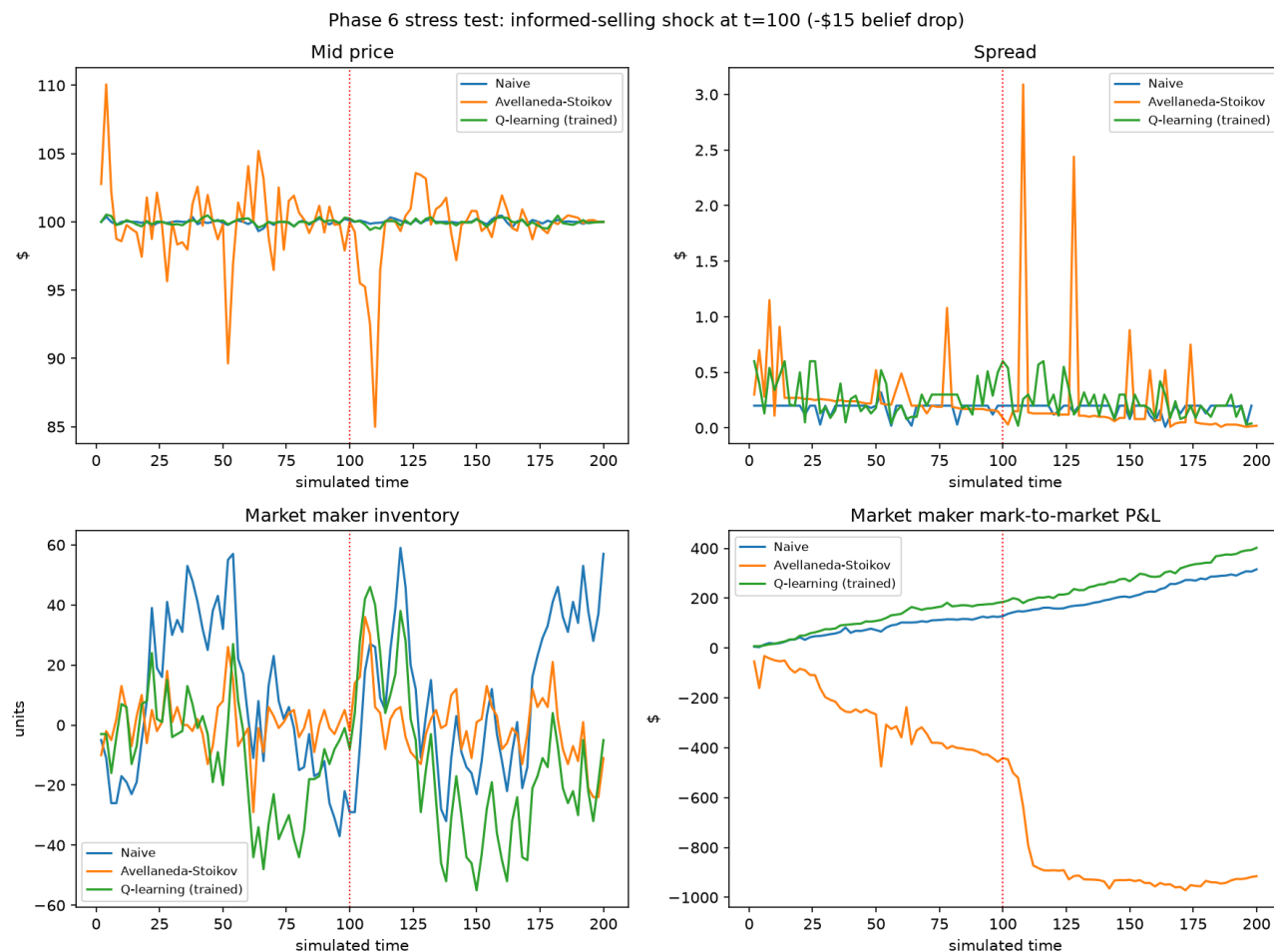
Scenario Design

The same five-noise-trader, one-informed-trader order flow used throughout the project runs for two hundred time units with one market maker at a time. At time one hundred, the informed trader's belief is knocked down by fifteen dollars, simulating unexpected bad news, which is implemented by calling the event loop's run-until method twice in succession with the perturbation applied directly to the agent between the two calls. No new simulator machinery was required for this, since it's pure orchestration of what Phases 1 through 5 already built. Everything downstream of the perturbation, including the resulting burst of aggressive informed selling as its signal diverges from the market's still-stale quotes, and how far the price actually moves, is genuine emergent behaviour, not scripted.

AN HONEST CAVEAT ABOUT THE COMPARISON'S DESIGN

This is not a controlled experiment where the market path is held fixed and only the market maker's reaction is observed. Each market maker runs in its own, fully separate simulated market, and every other agent reacts to that market's own live mid price, so a different market maker genuinely produces a different emergent market, not simply a different profit-and-loss outcome layered on an identical price path. That's realistic, since a real market maker's own liquidity provision does shape how a shock propagates, but it means any claim about why one market's outcome differs from another's is offered here as a plausible mechanism, not an isolated, statistically proven cause.

Results



Mid price, spread, inventory, and mark-to-market P&L for all three market makers under the identical shock.

Market maker	Min mid price	Max spread	Inventory (final)	P&L (final)
Naive	\$99.32 (before the shock)	\$0.32	+57 (over its 50-unit cap)	\$316
Avellaneda-Stoikov	\$84.99 (a real 15% crash)	\$3.09	-11	-\$915
Q-learning (trained)	\$99.41	\$0.60	-5	\$402

Naive: price-stable, but risk-blind

Its fixed spread does not adapt to anything, so it mechanically kept absorbing the burst of informed selling at a stable price. Its market's mid price never even dipped as far during the shock as it had from ordinary noise trading over thirty time units earlier. But stable price is not the same as safe. Inventory ended at plus fifty-seven, seven units past its own configured cap, through the same overshoot mechanism first noted in Phase 2, where the limit checks whether to quote but not how much to quote by. It also finished the most profitable of the three in this specific run, which is closer to luck than to skill, since it accumulated a large long position at falling prices during the crash and the price happened to recover by the end of the run. A single run's profit outcome for an inventory-blind strategy should not be read as evidence that it handled the shock well.

Avellaneda-Stoikov: good inventory control, bad outcome

Its market saw a genuine, deep crash, a real fifteen percent move to \$84.99, and its own spread spiked to \$3.09, over ten times its typical level. This is close to the opposite of what optimal market making is supposed to buy a market. The likely mechanism is that its reservation price re-centres on the live mid price every requote, and during the crash the mid itself was gapping violently, so each new quote chased an already-moving target rather than anchoring it. Per the caveat above, that thinner, chasing liquidity is a plausible contributor to the mid gapping as far as it did in this particular market. Its own profit and loss reflects this badly, down to minus \$915 by the end of the run and still falling. Its inventory management, taken on its own, actually worked, since it stayed close to flat throughout, between minus two and minus eleven units, but that discipline did not translate into good financial outcomes here, undercutting the theory's implicit promise that managing inventory well is sufficient.

Q-learning: the most balanced outcome

Its market's price dipped only slightly, to \$99.41. Its spread widened moderately, to \$0.60, more than the naive strategy but nowhere close to Avellaneda-Stoikov's spike. It took on meaningful inventory during the burst, peaking near forty-six units, close to its own fifty-unit cap, so it was not simply dodging the flow, and it finished with the best profit and loss of the three, still climbing. It was never given the Avellaneda-Stoikov formula, a crash-detection rule, or any explicit notion that this was a stress scenario at all. It learned a policy against ordinary order flow in Phase 5 that happened to generalize reasonably well to a scenario it had never seen.

Summary

The naive market maker survived on price stability but not on risk discipline. The theoretically optimal Avellaneda-Stoikov agent kept tight inventory control but presided over the worst price dislocation and the worst profit-and-loss outcome of the three, a reminder that a model's theoretical optimality is only as good as its calibration and its assumptions (constant volatility, no explicit adverse-selection term) continuing to hold in the scenario it actually faces. The trained Q-learning agent, with the least theoretical justification of the three going in, produced the most balanced outcome: contained price impact, controlled but not zero inventory, and the best profit and loss.

8

Conclusions

Framed for a Trading / Quant Research Interview

What Drives Spreads, In This Simulator

There are two distinct mechanisms, and it matters which one a given market maker actually has. First, there's order-flow composition. Spread widens with informed-trading intensity even for a market maker with no explicit adverse-selection pricing at all, as shown in Section 5, because faster quote depletion under heavier informed flow leaves a wider effective spread while the requote cycle catches up. That's a second-order, mechanical effect. Second, and far more powerful when present, is an explicit pricing model. Avellaneda-Stoikov's spread responds directly to time-to-horizon and a calibrated volatility parameter, and in the stress test its market showed a spread more than ten times wider than baseline during genuine turbulence, something no fixed-spread strategy could ever produce. The practical lesson is that a market maker's spread is only as responsive as the model that sets it. A fixed-spread strategy isn't "conservative" in any meaningful sense during a shock. It's simply blind to it.

Each Market Maker's Biggest Risk

Market maker	Biggest observed risk
Naive	No proactive risk management at all. Inventory grows unchecked until a hard cap is hit, and can even overshoot that cap on a single large fill, since the limit gates whether to quote, not how large a quote to size.
Avellaneda-Stoikov	Model risk. A formula calibrated on calm-market volatility, with no adverse-selection term of its own, degrades badly exactly when the market conditions it assumed stop holding, and good textbook inventory discipline did not prevent the worst P&L outcome under stress.
Q-learning	Generalization risk. It performed well in a stress scenario it was never trained on, but that's a favourable roll of a single run's random draw, not a guarantee, and its action grid is coarse enough (five spread choices, five skew choices) that it may be structurally unable to react as aggressively as a genuinely severe shock would require.

What I Would Change About the Model

- **Re-estimate Avellaneda-Stoikov's volatility parameter in real time**, rather than calibrating it once from a calm baseline. This is the single clearest lever for closing the gap between its theoretical promise and its stress-test performance.

- **Widen the Q-learning agent's action grid**, particularly its skew choices, to see whether it would learn to hedge more aggressively if allowed to, closing some of the magnitude gap with Avellaneda-Stoikov honestly rather than by construction.
- **Use a decaying learning rate** for Q-learning instead of a constant one, for a genuine fixed-point convergence guarantee rather than a policy that keeps slowly drifting even after aggregate performance has plateaued.
- **Give the informed trader's true-price process time-varying volatility**, such as a regime-switching or GARCH-style process instead of a constant one. The Phase 4 volatility-clustering result suggests this is the single missing mechanism standing between this simulator and a much more realistic return-series signature.
- **Validate against real LOBSTER tick data**, the brief's original optional stretch goal and the one piece of empirical rigor not yet attempted. Sanity-checking against a genuine market is a stronger portfolio claim than stylized facts checked against theory alone.
- **Run the stress test across multiple random seeds**, not just one illustrative run. The naive strategy's positive profit and loss in Section 7 looks like it could easily flip sign under a shock that doesn't recover by the end of the simulated session.

Closing

Every phase of this project produced a runnable artifact, not just a design document. There's a matching engine with a hundred and thirty-nine automated tests, a metrics dashboard generated from a real simulated session, three theory comparisons with numbers reproduced from a fresh run for this report, a trained reinforcement-learning agent committed to the repository as an actual model file, and a stress test whose conclusions were checked against real output rather than assumed. Several of the most useful findings in this report, including the negative-spread artifact traced to self-trade prevention, the false-alarm-then-fix story in the reinforcement-learning training, and Avellaneda-Stoikov's poor stress-test showing despite good inventory discipline, were not anticipated going in. They surfaced because the project was built to be interrogated at every stage, not just demonstrated once at the end.

A

Appendix

Technology Stack

- Python 3.14, managed with uv
- sortedcontainers, for the order book's price-level structure
- matplotlib, for every chart in this report
- pytest and pytest-cov, for the 139-test suite
- reportlab, for this report itself

Repository Structure

```
src/mm_sim/models.py: Order, Trade, enums
src/mm_sim/order_book.py: the matching engine
src/mm_sim/event_loop.py: the discrete-event simulator
src/mm_sim/agents/: all five agent types plus the shared QuotingAgent base
src/mm_sim/metrics.py: book snapshots, order impact, mark-to-market P&L
src/mm_sim/analysis.py: returns, autocorrelation, volatility estimation
examples/: seven runnable demonstration and analysis scripts
models/: the trained Q-learning agent, committed as a real artifact
tests/: 139 tests across sixteen files
DESIGN.md, DECISIONS.md, ANALYSIS.md, README.md: the project's own running
documentation
```

Reproducing These Results

```
uv sync
uv run pytest
uv run python examples/run_simulation.py
uv run python examples/plot_metrics.py
uv run python examples/phase4_analysis.py
uv run python examples/train_rl_market_maker.py
uv run python examples/compare_rl_vs_avellaneda_stoikov.py
uv run python examples/phase6_stress_test.py
```

Every number and every chart in this report was reproduced from a fresh run of these commands immediately before the report was finalized.

Repository

github.com/tobiedwards01/marketmicrostructuresim